COP2034C Module 9 Programming Assignment

Instructions

In this assignment, we will continue developing our Palmer Penguins program from Module 8. Previously, we used lists and dictionaries to organize penguin observations and calculate summary statistics.

In Module 9, we will introduce object-oriented programming by creating a Python class that represents an individual penguin observation. Instead of working exclusively with separate variables, we will organize related information into objects containing attributes and methods.

We will also learn how to create a custom Python module in Google Colab and import that module into our existing program.

We will practice selected Chapter 9 concepts:

- Defining a class using the class keyword.
- Creating a constructor using __init__().
- Using self to define and access instance attributes.
- Creating instance methods.
- Instantiating objects from a class.
- Creating and importing a custom Python module.

Continue working incrementally. Make one change, run the program, verify the results, and then proceed.

1. Start with Your Module 8 Solution

Open your completed Module 8 notebook in Google Colab and make a copy for Module 9. Suggested notebook name:

```
PalmerPenguinsM9.ipynb
```

The Python program you eventually download should be named:

```
PalmerPenguinsM9.py
```

Update the ID header:

```
# PalmerPenguinsM9.py
# Your Name
# Submission Date
# Analyze Palmer Penguin observations using classes and objects
```

Retain the working functionality from Module 8, including the existing functions, species statistics, body mass summary, dictionary operations, and interactive species exploration.

We will add the new object-oriented functionality without removing the earlier demonstrations of Python concepts.

2. Create a Penguin Class

A class defines the attributes and behaviors shared by a group of objects. In our dataset, each observation describes an individual penguin, making a Penguin class a natural way to organize that information.

We will define the class in a separate Python module named **penguin.py**. In Google Colab, create a new code cell and enter:

```
%%writefile penguin.py
class Penguin:
    def __init__(self, sample_number, species, body_mass):
        self.sample_number = sample_number
        self.species = species
        self.body_mass = body_mass
```

Run the cell.

- The %%writefile command is a Google Colab/IPython cell magic that writes the contents of the cell to a file. In this case, it creates penguin.py in the current working directory.
- The __init__() method initializes a new Penguin object. The self parameter refers to the particular object being created.

Each object will have three instance attributes:

Attribute	Description
sample_number	Observation number from the CSV file
species	Penguin species name
body_mass	Penguin body mass in grams

For this assignment, we will use only three attributes to keep the class simple.

3. Add an Instance Method

Classes can also contain methods, which are functions associated with objects.

Add a method named display_info() to the Penguin class. This method should display the sample number, species, and body mass of the current object. Use the following structure:

```
def display_info(self):
```

```
        # display the sample number
        # display the species
        # display the body mass
```

Replace the comments with appropriate print() statements using f-strings.

Remember that display_info() must be indented inside the Penguin class, at the same level as __init__().

After updating the class, run the %%writefile cell again to overwrite penguin.py with the completed definition.

4. Import the Custom Module

We can now use our Penguin class in the main Palmer Penguins program. Near the top of PalmerPenguinsM9.py, after the ID header and existing import statements, add:

```
from penguin import Penguin
```

This statement imports the Penguin class from the custom module named penguin.py. The class can now be used in our program just like classes provided by Python's standard library.

Important: Run the %%writefile cell before running the main program so that penguin.py exists. If you modify the class later, rerun the %%writefile cell and restart the Colab runtime before testing the changes. Remember to upload the CSV file again if necessary.

5. Create a Penguin Object

Inside main(), after the existing body mass summary and species dictionary output, create a new section demonstrating the Penguin class. First, retrieve the first observation from the CSV file:

```
penguin_row = read_penguin_row(1)
penguin_data = penguin_row.split(",")
```

Next, retrieve the appropriate fields from penguin_data and convert numeric values to integers where necessary. Create a Penguin object using the values you retrieved and the following structure:

```
sample_penguin = Penguin(
    # add the sample number, species, and body mass
)
```

Replace the comments with the appropriate values from penguin_data. Do not hard-code the observation values. They must come from the CSV file. After creating the object, call its display_info() method:

```
print("\nPenguin Object Demonstration")
```

```
sample_penguin.display_info()
```

Notice that the object stores its own attributes and uses its own method to display them.

6. Create a List of Penguin Objects

In Module 8, we created a list containing the body masses of five observations. Now we will create a list containing five Penguin objects. Use the existing sample_rows list:

```
sample_rows = [1, 2, 3, 4, 5]
```

Create an empty list:

```
penguin_objects = []
```

Write a for loop that processes each row number in sample_rows. For each observation:

1. Retrieve the CSV row using read_penguin_row().
2. Split the row into individual values.
3. Create a Penguin object using the sample number, species, and body mass.
4. Append the object to penguin_objects.

Use this structure as a guide:

```
for row_number in sample_rows:
    # retrieve and split the CSV row
    # create a Penguin object
    # append the object to penguin_objects
```

Use the loop to create the objects (do not create five separate variables for the five penguins). After completing the loop, display the objects by calling their instance methods:

```
print("\nPenguin Objects")
for penguin in penguin_objects:
       penguin.display_info()
```

This demonstrates that multiple objects created from the same class can contain different attribute values.

7. Calculate an Average Using Object Attributes

We previously calculated average body mass using a list of integers. Now we will calculate the same statistic using the attributes stored in our Penguin objects.

- Create a variable named total_object_mass and initialize it to zero.
- Use a for loop to add each object's body_mass attribute to the total.
- Calculate the average by dividing the total by the number of objects.

Display the results:

```
print("\nObject-Based Body Mass Summary")
print(f"Observations: {len(penguin_objects)}")
print(f"Total body mass: {total_object_mass} g")
print(f"Average body mass: {average_object_mass:.2f} g")
```

Compare these results with the body mass summary calculated in Module 8. They should match since both calculations use the same five observations.

This is an important distinction: in Module 8, we accessed values directly from a list of integers. In Module 9, we are accessing those values through attributes belonging to objects.

8. Organize the New Code in main()

Place the new Module 9 logic after the existing species dictionary output and before the interactive species exploration. Your program should follow this general sequence:

```
main()
    display species statistics
    process two sample observations
    collect and summarize body masses
    display species dictionary

    create one Penguin object
    display the object's information

    create a list of five Penguin objects
    display each object
    calculate object-based body mass statistics

    explore species using the existing while loop
    display final exploration summary
```

Keep the original Module 8 functionality working. The new class and object demonstrations are additions to the program.

Expected Output

The program should continue displaying all the information from Module 8. The new Module 9 output should include the following sections.

```
Penguin Object Demonstration
Sample Number: 1
Species: Adelie
Body Mass: 3750 g

Penguin Objects
```

```
Sample Number: 1
Species: Adelie
Body Mass: 3750 g
Sample Number: 2
Species: Adelie
Body Mass: 3800 g
Sample Number: 3
Species: Adelie
Body Mass: 3250 g
Sample Number: 4
Species: Adelie
Body Mass: 3450 g
Sample Number: 5
Species: Adelie
Body Mass: 3650 g

Object-Based Body Mass Summary
Observations: 5
Total body mass: 17900 g
Average body mass: 3580.00 g

Explore the Palmer Penguins
Enter Chinstrap, Gentoo, Adelie, or Done.
```

The output above uses the provided Palmer Penguins CSV file. Minor whitespace differences are acceptable as long as the information is correct and readable.

Test Your Program

Before submitting, verify that:

- The original Module 8 functionality still works correctly.
- The penguin.py module contains the Penguin class, its constructor, and the display_info() method.
- The main program successfully imports the custom module.
- A Penguin object is created using values retrieved from the CSV file.
- A list contains five separate Penguin objects.
- The program calls display_info() for each object.
- The object-based body mass calculations match the Module 8 results.
- The existing interactive species exploration still works, and entering Done ends the loop.

Data Science Connection

In Module 8, we used lists and dictionaries to organize penguin observations and perform statistical calculations. In Module 9, we introduce a way to organize data using objects.

In data science, an observation often contains several related measurements. For example, one penguin observation contains a sample number, species, body mass, bill measurements, and other attributes. A class provides a way to group these related values into a single object. Our Penguin class currently stores only three attributes, but the same approach could be expanded to represent additional measurements. Objects can also contain methods that operate on their data. This helps organize larger programs by keeping related information and operations together.

Our new program demonstrates an object-oriented data-processing workflow:

- Retrieve observations from a CSV file.
- Convert individual values into appropriate Python data types.
- Create objects representing individual observations.
- Store multiple objects in a list.
- Access object attributes to calculate summary statistics.

Separating the Penguin class into its own Python module also demonstrates modularity, an important software development practice. As data science applications grow, developers frequently organize related classes and functions into reusable modules. For example, one module might define data structures, another might process datasets, and another might perform statistical calculations. Although our example uses only five observations, the same organizational approach can be applied to much larger datasets.

Readability

Continue following the coding conventions established in previous modules:

- Use meaningful variable, class, and method names.
- Use consistent four-space indentation.
- Include comments describing important operations.
- Use logical blank lines to separate program sections.
- Keep the class definition in the custom module and the main program logic in PalmerPenguinsM9.py.

The Penguin class name follows Python's convention of using capitalized class names, while display_info() follows the lowercase-with-underscores convention for method names.

Screenshot and Submission

Run the completed program in Google Colab. Capture enough of the execution output to demonstrate the new Penguin object functionality, object-based body mass summary, and existing interactive species exploration.

Download the following files from Google Colab:

1. PalmerPenguinsM9.py: download the main program as a Python file. Make sure the **%%writefile penguin.py** command **is not** included in the exported program, it is Colab-specific and not valid Python syntax.

2. penguin.py: open the Files panel on the left side of Colab. Locate penguin.py, click the three-dot menu next to the filename, and select Download. Add an ID header before submitting if you have not already done so, e.g.,

```
# penguin.py
# Your Name
# Submission Date
# Class representing a penguin
```

3. PalmerPenguinsM9.png: save a screenshot of the program's execution output.

Upload all three files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.